

AI-Powered Agentic Software Engineering Platform

Project Context & Product Specification

Contents

1. Problem1

2. Product Vision1

3. Feature List2

- 3.1 GitHub & Repository2
- 3.2 AI Analysis2
- 3.3 Testing & Quality3
- 3.4 Issues & Pull Requests4
- 3.5 Agentic Development4
- 3.6 Context Continuity / Memory5
- 3.7 Agent Safety & Control6
- 3.8 Traceability7
- 3.9 Dashboard / Observability7

4. Context Continuity Engine7

- 4.1 Core Idea7
- 4.2 What the System Should Remember7

5. Agentic System8

- 5.1 Core Capability Loop8
- 5.2 Agent Responsibilities8

1. Problem

Modern software teams juggle code review, testing, debugging, issue tracking, and pull request management across disconnected tools. Each of these activities happens in isolation — a reviewer flags a bug, a tester writes a case, a developer fixes it — with no shared thread connecting why the work exists, what was already tried, and what happened last time.

This fragmentation gets worse with AI involvement. Existing AI coding assistants are stateless: every session starts from zero. They don't know the project's architecture, the decisions that shaped it, the rules the team follows, or the approaches that already failed. Developers end up re-explaining context over and over, and AI-generated changes often ignore conventions or repeat mistakes because nothing was remembered.

At the same time, giving an autonomous agent free rein over a codebase is unsafe. Without permissions, sandboxing, and human checkpoints, agentic execution risks unreviewed changes reaching production, runaway loops, or actions no one can trace back to a reason.

The problem this platform solves: software engineering work is fragmented, AI assistance is forgetful, and autonomous agents lack the guardrails needed for developers to trust them. Teams need a single system that understands a codebase, remembers its engineering context over time, and lets an agent do real engineering work — safely, transparently, and with a human always in the loop before anything merges.

2. Product Vision

An AI-powered Agentic Software Engineering Platform that integrates with GitHub to understand a project's codebase, remember its engineering context, analyze changes, execute development tasks safely, validate the results, and prepare changes for human approval.

Who it's for: software development teams who want AI to do more than comment on pull requests — teams who want an agent that can pick up a task, do the work, and hand back a verified, reviewable pull request.

What it is: not just an AI code-review tool. It connects the full engineering loop — Requirement → Codebase → Analysis → Planning → Implementation → Testing → Verification → Draft PR → Human Review → Merge → Project Memory — into one continuous, observable process.

What makes it different:

- **It remembers.** Architecture, decisions, rules, past attempts, and failures persist across sessions instead of vanishing when a conversation ends.
- **It acts, not just advises.** An agent can plan, implement, test, and self-verify — not only suggest.
- **It stays safe by design.** Every agent action is scoped by permissions, sandboxed, budgeted, and reversible, and nothing merges without a human.
- **It stays traceable.** Every change can be followed back to the requirement that motivated it and the evidence that justified it.

What the experience should feel like: a developer hands the platform a task or an issue. The platform already knows the project — its structure, its history, its rules, what's been tried before. It plans the work, shows the plan for approval, does the work in an isolated environment, tests and verifies its own output, and returns a draft pull request with clear evidence of what it did and why. The developer reviews, and either merges, asks for changes, or rolls back — and the platform remembers the outcome for next time.

3. Feature List

Each feature below is tagged with a delivery priority: **P0** core to the first release, **P1** important but not blocking, **P2** future scope.

3.1 GitHub & Repository

GitHub Integration **P0**

Purpose: The platform works alongside GitHub rather than replacing it, so teams keep their existing workflows.

What it does: Connects to GitHub for authentication, repository and branch access, issues, commits, git history, CI results, and draft PR creation with status updates.

Repository Engine **P0**

Purpose: AI and agents need real understanding of a project — not just the changed lines — to give relevant analysis and safe changes.

What it does: Builds practical understanding of repository structure, files, modules, functions, dependencies, configuration, APIs, tests, and how components relate to each other.

3.2 AI Analysis

AI Code Review P0

Purpose: Surface correctness, quality, and architectural concerns before they become problems.

What it does: Reviews changes for correctness, maintainability, code quality, bugs, bad practices, performance, and architecture, with severity, location, reasoning, and suggested action for each finding.

Bug Detection P0

Purpose: Catch likely defects that a human reviewer under time pressure might miss.

What it does: Flags logical errors, null/undefined risks, incorrect conditions, edge cases, exception-handling gaps, race conditions, bad state transitions, and regression risk.

Basic Security Analysis P0

Purpose: Give teams a first line of defense against common, high-impact security mistakes without trying to replace a dedicated security product.

What it does: Flags hardcoded secrets, unsafe input handling, injection risk, auth/authorization issues, insecure configuration, and risky dependency usage.

Change Impact / Blast-Radius Analysis P0

Purpose: Help developers and agents understand what a change might break before it ships — a strong differentiator versus line-by-line review.

What it does: Traces a change from the file and function level through dependent components and affected APIs/services, recommends the tests most likely to be relevant, and produces an overall risk assessment.

Code Explanation P0

Purpose: Reduce the time developers spend deciphering unfamiliar code, PRs, errors, and failures.

What it does: Explains selected code, functions, architecture, PR changes, review findings, error messages, and test failures in plain language.

Repository-Aware AI Chat P1

Purpose: Let developers ask questions about their own codebase and get grounded answers instead of generic ones.

What it does: A chat interface that answers questions about the repository using real repository context and project memory.

3.3 Testing & Quality

Test Generation P0

Purpose: Ensure changes are covered by relevant tests without relying on developers to remember every case.

What it does: Generates unit, integration, regression, edge-case, and negative tests, informed by the changed code and the project's existing testing conventions.

Test Execution & Result Analysis P0

Purpose: Give a clear, trustworthy signal on whether a change is actually safe to review.

What it does: Runs generated and existing tests in a controlled environment, reports pass/fail status across suites, and — on failure — investigates logs to identify the failing test, related code, and a probable cause.

CI Failure Diagnosis P0

Purpose: Turn a raw CI failure into an understandable, actionable explanation rather than a wall of logs.

What it does: Reads CI failure logs, identifies the failing test, locates related code, determines the root cause, and proposes or applies a fix before re-running.

Independent Verification P0

Purpose: Avoid the conflict of interest in letting the same agent that wrote code also be the sole judge of whether it's correct.

What it does: A separate verification pass that challenges the implementation, checks edge cases, re-checks the original requirement, and reviews the tests before anything is marked ready.

Agent Quality Score P1

Purpose: Give reviewers a fast, structured signal on the quality of an agent's work, without replacing their judgment.

What it does: Produces a scorecard (requirement match, implementation, test coverage, code quality, security, overall) to assist — not automatically approve — human review.

Project Standards Engine P0

Purpose: Encode a team's engineering expectations — separate from historical decisions — so agent output consistently meets them.

What it does: Stores standing rules such as required test coverage, architectural conventions, security baselines, and commit/PR conventions, and checks agent work against them.

3.4 Issues & Pull Requests

Issue Creation P0

Purpose: Turn AI findings into actionable, trackable work instead of comments that get lost.

What it does: Converts an AI finding into a GitHub Issue with title, description, severity, affected files, evidence, suggested fix, and any related PR.

Issue → Automatic Draft PR Agent P0

Purpose: Close the loop from a reported problem or request to a reviewable solution.

What it does: Takes a GitHub issue, understands the requirement, retrieves relevant context, locates the relevant code, plans the implementation, executes it, tests and verifies it, and opens a draft pull request for human review.

3.5 Agentic Development

Agentic Software Engineering System P0

Purpose: Extend the platform beyond passive review into an agent that can actually do engineering work.

What it does: Understands a task, retrieves project context, plans the work, seeks approval where required, works in an isolated workspace, modifies code, generates/updates tests, runs and diagnoses tests, retries within limits, verifies the result, opens a draft PR, and records the outcome in memory.

Natural-Language Development Tasks P0

Purpose: Let developers hand off work the way they'd describe it to a teammate, not through rigid task forms.

What it does: Accepts plain-language requests (e.g. “add pagination to the users API”) and converts them into a structured execution plan.

Agent Task Planner P0

Purpose: Make agent behavior predictable and reviewable before any code changes, rather than a black box.

What it does: Produces a step-by-step plan for a task that the developer can approve or edit before execution begins.

Agent Execution Sandbox P0

Purpose: Ensure autonomous work can never directly affect a developer's machine or the live repository.

What it does: Runs all agent changes, commands, and tests inside a temporary, isolated workspace, then collects results and discards the workspace afterward.

Agent Permission / Policy Engine P0

Purpose: Ensure an agent's authority matches the risk of the action it's taking.

What it does: Enforces tiered permission levels (read, workspace edits, git operations, GitHub actions, human-gated merge) with per-action policies of allow, ask, or deny.

Agent Self-Verification Loop P0

Purpose: Require the agent to demonstrate correctness rather than simply asserting it.

What it does: After making a change, the agent runs tests, diagnoses and fixes failures, re-runs, and only proceeds to a draft PR once it has evidence the work is correct.

Agent Task Queue P2

Purpose: Let developers hand off multiple pieces of work without waiting on each one sequentially.

What it does: Maintains a queue of pending agent tasks and schedules their execution.

3.6 Context Continuity / Memory

Context Continuity Engine P0

Purpose: The core differentiator — an agent should never have to relearn a project from scratch.

What it does: Preserves important project knowledge across conversations, agent runs, context-window limits, task handoffs, failed attempts, completed tasks, and future sessions, extracting what matters rather than storing everything.

Four Levels of Context P0

Purpose: Organize memory so the right information is available at the right scope — task, session, project, or experience.

What it does: Structures context into working context (current task/conversation), session context (current run and checkpoints), project memory (architecture, decisions, rules), and experience memory (past failures, solutions, history).

Project Memory P0

Purpose: Give the agent a standing understanding of what the project is.

What it does: Stores the project's purpose, architecture, technologies, services, entry points, database structure, APIs, important modules, and external integrations.

Decision Memory P0

Purpose: Prevent the agent from proposing approaches the team has already considered and rejected.

What it does: Records important decisions — what was chosen, why, what was rejected, the impact, and current status — so past reasoning stays available.

Task Memory P0

Purpose: Allow work to be picked back up exactly where it left off, across sessions.

What it does: Tracks a task's goal, status, completed steps, remaining steps, and working branch.

Agent Work Memory P0

Purpose: Preserve a factual record of what an agent actually did, for continuity and review.

What it does: Logs the actions taken during an agent run, files modified, results, and the resulting PR.

Failure Memory P1

Purpose: Stop the agent from repeating approaches that are already known not to work.

What it does: Records failed attempts, why they failed, what to avoid, and what to prefer instead.

Project Rules **P0**

Purpose: Encode standing engineering constraints the agent must always respect.

What it does: Stores rules such as required testing, prohibited automatic production changes, and PR-only merge behavior, for the agent to check itself against.

Context Checkpoints **P0**

Purpose: Let long-running agent work survive context limits without losing important state.

What it does: Periodically extracts important information from the current context into a durable checkpoint (objective, completed steps, current state, next step, important constraints, relevant files) that a new context can resume from.

Context Retrieval **P0**

Purpose: Keep the agent focused by surfacing only what's relevant, rather than overwhelming it with the full memory store.

What it does: Ranks and compiles relevant project memory, decisions, related tasks, recent changes, known failures, and rules into the agent's working context for a given task.

Context Conflict Detection **P1**

Purpose: Prevent stale memory from misleading the agent when the project has moved on.

What it does: Detects when stored memory contradicts the current repository state and prompts the developer to choose which should be trusted.

Memory Versioning **P1**

Purpose: Keep the memory store internally consistent as the project evolves.

What it does: Tracks lifecycle status of memory entries (active, superseded, archived, invalid) so outdated entries don't conflict with current ones.

Sensitive Context Protection **P0**

Purpose: Ensure sensitive data never ends up persisted in project memory.

What it does: Detects and redacts passwords, API keys, access tokens, secrets, credentials, and sensitive environment variables before anything is stored.

Git + Memory Correlation **P1**

Purpose: Combine what changed with why it changed for a complete picture.

What it does: Unifies repository context, git history, and the memory engine into a single correlated context for the agent.

Memory Dashboard **P1**

Purpose: Give developers visibility into and control over what the system remembers.

What it does: Displays stored architecture, decisions, rules, active/completed tasks, known failures, and checkpoints, with controls to inspect, correct, delete, archive, mark obsolete, pin, and view the source of each memory item.

3.7 Agent Safety & Control

Agent Rollback & Recovery **P0**

Purpose: Ensure agent work is never a one-way door.

What it does: Creates recoverable checkpoints throughout an agent run so a developer can stop the agent, restore a prior checkpoint, discard changes, or resume from a checkpoint.

Agent Kill Switch **P0**

Purpose: Give developers an immediate, unconditional way to stop agent activity.

What it does: Halts a running agent on command, saving a checkpoint and preserving task state before stopping.

Agent Budget & Resource Limits P0

Purpose: Prevent runaway agent loops from consuming unbounded time, iterations, or cost.

What it does: Enforces configurable limits on iterations, execution time, test retries, tool calls, and token/cost, stopping and checkpointing the agent with a prompt to the developer if a limit is hit.

Human Approval Checkpoints P0

Purpose: Keep a human in control of the agent by default, rather than fully autonomous execution.

What it does: Requires human approval at key points — notably before implementation begins and before a PR is merged.

Cross-Agent Handoff P2

Purpose: Lay the groundwork for a future multi-agent architecture without committing to its complexity now.

What it does: Defines how specialized agents (planning, coding, testing, review) would exchange structured context rather than full conversations.

3.8 Traceability

Requirements → Code → Test → PR Traceability P0

Purpose: Let anyone answer “why does this code exist” by tracing it back to the requirement that motivated it — a major software-engineering differentiator.

What it does: Connects a requirement or user story through the GitHub issue, agent task, changed files, tests, pull request, and verification outcome into one traceable chain.

Agent Decision / Evidence Trace P1

Purpose: Make agent actions explainable and auditable without exposing raw internal reasoning.

What it does: For significant actions, presents a concise summary of the task, the evidence behind the decision, a confidence level, and the files affected.

3.9 Dashboard / Observability

Agent Activity Timeline P0

Purpose: Make every agent run auditable, step by step.

What it does: Logs a chronological record of what the agent did during a run — from task receipt through analysis, implementation, testing, correction, and PR creation.

Agent Analytics P2

Purpose: Give teams visibility into agent performance and reliability over time.

What it does: Tracks metrics such as tasks completed, success rate, draft PRs, average iterations and execution time, test pass rate, human rejection rate, rollback rate, and cost.

Minimal Developer Dashboard P0

Purpose: Give developers one place to see and act on platform activity.

What it does: A lightweight dashboard surfacing repository status, active agent tasks, recent findings, and pending reviews.

4. Context Continuity Engine

4.1 Core Idea

The agent should continue work with the important context of the project instead of starting from zero every time.

This is the platform's central differentiator. Instead of treating every session as a blank slate, the system maintains a living memory of the project that the agent draws on for every task.

4.2 What the System Should Remember

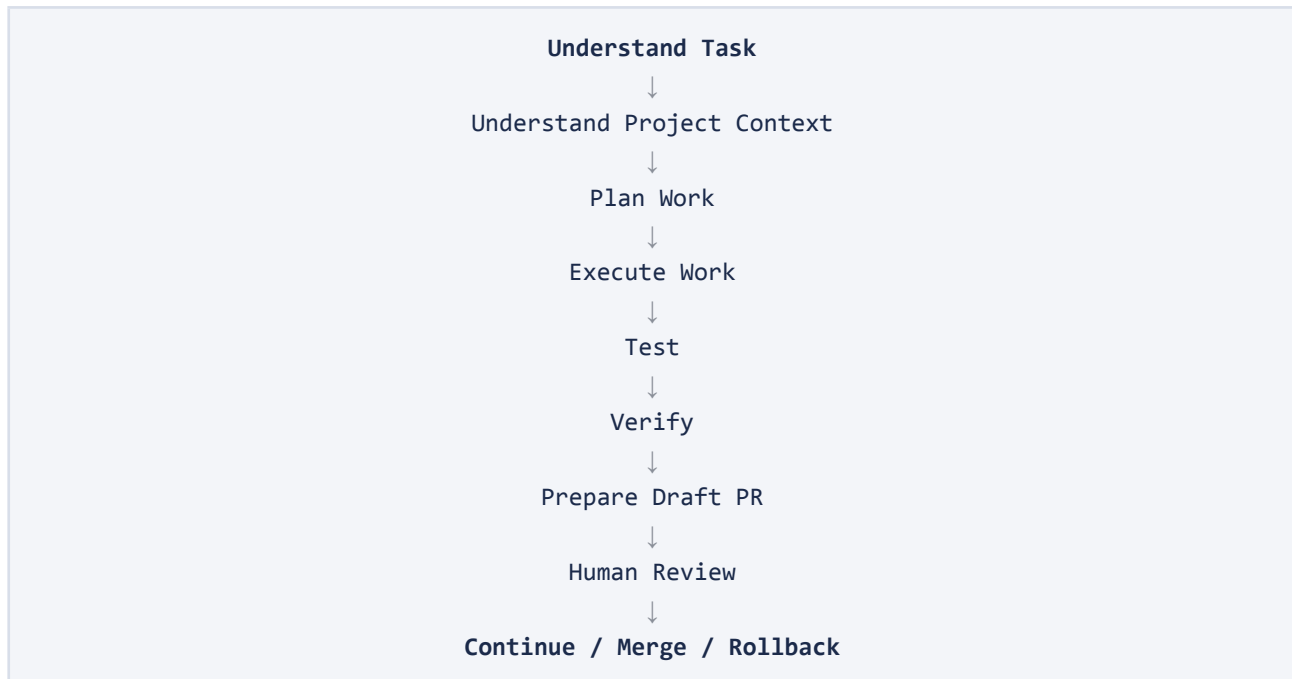
- **Project context** — the project's purpose, architecture, technologies, services, and how its major pieces fit together
- **Architecture knowledge** — structure, entry points, database design, APIs, important modules, and external integrations
- **Important decisions** — what was decided, why, what was rejected instead, and what depends on that decision
- **Project rules** — standing engineering constraints the team has set (testing requirements, what the agent may never touch automatically, PR-only merge behavior)
- **Current tasks** — what's in progress, what's been completed so far, and what remains
- **Previous agent work** — what an agent actually did in prior runs: actions taken, files touched, results, and resulting PRs
- **Failed approaches** — attempts that didn't work, why they failed, and what to do instead
- **Checkpoints** — extracted state from long-running work so it can resume without losing important progress
- **Conversation context** — the current task and discussion, kept available for the duration of the working session
- **Relevant development history** — the connective tissue between git history and the reasons behind changes

Memory is not a raw transcript log. The system should extract and retain useful project knowledge rather than blindly storing every token exchanged, keep that knowledge organized by relevance and recency, detect when memory has gone stale against the current state of the repository, and give developers visibility into — and control over — what's remembered.

The result: an agent picking up a task already knows what the project is, what's been tried, what's been decided, and where the last attempt left off.

5. Agentic System

5.1 Core Capability Loop



5.2 Agent Responsibilities

The agent is the platform's major extension beyond passive AI code review. Given a task — whether a natural-language request or a GitHub issue — the agent should be able to:

1. **Understand the task** — interpret what's being asked, whether phrased as plain language or as a formal issue
2. **Understand project context** — pull in the relevant project memory, decisions, rules, and history needed to do the work correctly
3. **Plan the work** — produce a clear, step-by-step plan the developer can review, edit, or approve before anything changes
4. **Execute the work** — make the required changes inside an isolated environment, never directly against a developer's machine or live systems
5. **Test** — generate and/or update tests appropriate to the change, and run them
6. **Verify** — confirm the result actually meets the requirement, ideally through an independent check rather than the same agent grading its own work
7. **Prepare a draft PR** — package the verified change into a draft pull request with supporting evidence, never merging automatically
8. **Human review** — hand control back to a developer, who has full visibility into what was done and why
9. **Continue / Merge / Rollback** — based on human judgment, the work is merged, sent back for further iteration, or rolled back to a prior checkpoint — with the outcome recorded back into project memory either way

Throughout this loop, the agent operates within permission levels, budget limits, and mandatory human checkpoints, so autonomy is always bounded and every action can be traced back to a reason.